Specification–Implementation Balance: A Diagnostic Framework for AI-Mediated Software Construction

Egor Merkushev[1]

[1] *Independent Researcher*

ABSTRACT

We introduce a non-normative diagnostic framework for observing the balance between specification (intent), implementation (code), and AI inference cost in modern AI-assisted software development. Rather than proposing optimization targets or productivity metrics, the framework is designed to expose structural and economic shifts in the process of intent-to-implementation conversion.

We further extend the base Specification–Implementation Balance (SIB) formalism in three directions. First, we define a pre-implementation layer for situations where code does not yet exist, but specification structure, assumptions, and expected implementation complexity are already observable. Second, we introduce process and structural observability metrics that characterize verification pressure, friction, stability, cognitive load, and change propagation. Third, we propose retrospective defect metrics that make it possible to trace late-discovered failures or value misalignments back to their earlier specification roots. Taken together, these metrics form a longitudinal observability stack for AI-mediated software construction.

Keywords: software engineering — specification-driven development — AI-assisted programming — software metrics

INTRODUCTION

The increasing use of large language models in software development has blurred the boundary between specification and implementation. Code is no longer written exclusively by humans, while specifications increasingly act as executable or semi-executable artifacts. This shift challenges traditional code-centric metrics such as LOC, velocity, or test coverage.

In this work, we propose a set of observability metrics that characterize *where semantic intent resides* and *how it is transformed into executable systems*. The resulting framework is explicitly diagnostic rather than normative: it is intended to support observation, comparison, and qualitative reasoning, not optimization targets or human evaluation.

The central idea is that the balance between specification and implementation can be used as a foundational observable. From this balance, additional diagnostic layers follow naturally: pre-implementation observables when no code yet exists, process observables once intent begins to convert into executable artifacts, and retrospective defect observables when late discoveries reveal that earlier assumptions, specifications, or implementations were unsound.

1. ANALYTIC SCOPE AND NOTATION

We use two complementary viewpoints.

At the project level, we reason in terms of aggregate quantities: the amount of explicit specification, implementation, and observable functionality.

At the local level, we reason over a graph of nodes. A node $u \in V$ may represent a requirement, feature step, architectural decision, subsystem, or implementation unit, depending on the phase of work. Edges express relevant relations such as refinement, dependency, influence, or causal progression. Time is denoted by t, and $\varepsilon > 0$ is a stabilizer constant.

When we write $\mathcal{N}(u)$, we refer to a chosen *neighborhood* around node u used for local gradient computations, such as the cognitive load gradient. The embedding $x(u) \in \mathbb{R}^2$ denotes an optional geometric placement of nodes on a two-dimensional map for visualization. The symbol $\mathrm{dist}(u, v)$ denotes the graph distance between nodes u and v using selected edge types.

Throughout the paper we employ weighting coefficients such as $\alpha, \beta, \gamma, \delta, \lambda$. These coefficients are not universal constants but rather calibratable parameters that modulate the relative importance of different factors (e.g., regenerations versus rollbacks, boundaries versus modes, or inference costs) within a given project or orga-

nization. Selecting appropriate values for these weights is itself an empirical exercise and should reflect domain expertise and context-specific priorities.

2. ANALYTIC FRAMEWORK

2.1. *Specification as Intent Atoms*

We represent specification as a set of discrete *Intent Atoms*, denoted N_{spec}, composed of: user stories, acceptance criteria, invariants, and architectural decisions.

At node granularity, let $\mathcal{A}(u)$ denote the set of Intent Atoms attached to node u. We define the local specification signal as

$$S(u) \equiv |\mathcal{A}(u)|. \tag{1}$$

2.2. *Implementation as Functional Output*

Implementation is represented by two orthogonal quantities: (i) the observable functional output N_{func}, and (ii) the implementation size S_{impl}, measured in lines of code, tokens, or normalized AST units.

At node granularity, we denote by $I(u, t)$ a local implementation signal, for example a churn-weighted implementation delta or normalized change mass.

2.3. *AI Inference Footprint*

We define the inference footprint as the tuple $(C_{\text{inf}}, T_{\text{inf}}, N_{\text{calls}})$, corresponding to monetary cost, token volume, and non-tokenizable operations (e.g., external tool invocations, search queries, or discrete API actions).

2.4. *Terminology Conventions*

To avoid ambiguity, we explicitly separate two notions used throughout this paper.

Economic cost refers to monetary inference expenditure and is tied to C_{inf}. Metrics such as Intent Conversion Cost (ICC), Functional Cost (FC), and other cost-weighted variants intentionally retain the term *cost*.

Implementation effort refers to expected engineering work and realization complexity required to turn intent into stable software artifacts. In pre-implementation analysis this is proxied by $I^*(u)$ and related quantities (e.g., ΔI^* and defect-potential aggregates).

Unless explicitly marked as economic, realization-oriented quantities in this framework are described using the term *effort*.

2.5. *Premises and Assumptions*

Many development trajectories are driven not only by explicit intent, but also by assumptions about user value, architecture, integrations, constraints, or product direction. We therefore introduce a second local set,

$$\mathcal{P}(u), \tag{2}$$

the set of *Premise Atoms* associated with node u.

Premise Atoms express minimal assumptions such as: "users prefer this interaction pattern", "integration X is available", or "stakeholders accept this workflow".

We define the local premise signal as

$$P(u) \equiv |\mathcal{P}(u)|. \tag{3}$$

3. DERIVED METRICS

3.1. *Specification–Implementation Balance*

We define the Specification–Implementation Balance (SIB) as

$$\text{SIB} \equiv \frac{N_{\text{spec}}}{S_{\text{impl}}}. \tag{4}$$

This quantity characterizes the distribution of semantic intent between explicit specification and executable realization. No optimal value is assumed.

3.2. *Intent Yield*

The Intent Yield (IY) is defined as

$$\text{IY} \equiv \frac{N_{\text{func}}}{N_{\text{spec}}}. \tag{5}$$

IY captures how much observable functionality is produced per unit intent.

3.3. *Intent Conversion Cost*

The Intent Conversion Cost (ICC) is defined as

$$\text{ICC} \equiv \frac{C_{\text{inf}}}{N_{\text{spec}}}. \tag{6}$$

This metric reflects the economic cost of AI-mediated intent realization.

3.4. *Functional Cost*

The Functional Cost (FC) is given by

$$\text{FC} \equiv \frac{C_{\text{inf}}}{N_{\text{func}}}. \tag{7}$$

3.5. *Operational Yield*

The Operational Yield (OY) is defined as

$$\text{OY} \equiv \frac{N_{\text{func}}}{N_{\text{calls}}}. \tag{8}$$

OY captures how much observable functionality is produced per discrete non-tokenizable operation. It characterizes the efficiency of intent-to-implementation conversion through the lens of operational effort.

4. PRE-IMPLEMENTATION EXTENSION

The base SIB formalism is still meaningful before code exists, provided that implementation is replaced by an observable proxy for future realization cost. This allows diagnostic reasoning to begin in the specification phase, rather than only after executable artifacts are already present.

4.1. *Specification Verifiability*

Not all written specification contributes equally to confidence. Some Intent Atoms are operationalized through acceptance criteria, explicit checks, stakeholder confirmations, contradiction checks, or other forms of verifiability.

We therefore define the Spec Verifiability Coverage of node u as

$$N_{\text{ver}}(u) \equiv \sum_{a \in \mathcal{A}(u)} \mathbf{1}_{\text{ver}}(a), \tag{9}$$

the number of verifiable Intent Atoms in node u.

Then,

$$\text{SVC}(u) \equiv \frac{N_{\text{ver}}(u)}{|\mathcal{A}(u)|}. \tag{10}$$

This yields the effective specification signal

$$S_{\text{eff}}(u) \equiv S(u) \cdot \text{SVC}(u). \tag{11}$$

Given the definitions above, for nodes with $S(u) > 0$ we have $S_{\text{eff}}(u) \equiv N_{\text{ver}}(u)$. We retain the product form $S(u) \cdot \text{SVC}(u)$ to make explicit the decomposition into total specification mass and verifiability coverage. For nodes with $S(u) = 0$, $\text{SVC}(u)$ may be set to 0 by convention.

4.2. *Premise Verifiability*

Similarly, not all assumptions are equally trustworthy. Some premises are tested, confirmed, or explicitly ratified, while others remain implicit or weakly supported.

We define Premise Verifiability Coverage as

$$\text{PVC}(u) \equiv \frac{\sum_{p \in \mathcal{P}(u)} \mathbf{1}_{\text{check}}(p)}{|\mathcal{P}(u)|}. \tag{12}$$

This yields the effective premise signal

$$P_{\text{eff}}(u) \equiv P(u) \cdot \text{PVC}(u). \tag{13}$$

4.3. *Expected Implementation Potential*

Before implementation exists, local realization cost is not directly measurable. We therefore introduce an expected implementation potential,

$$I^*(u), \tag{14}$$

which serves as a proxy for the future mass of implementation implied by node u.

A generic form is

$$I^*(u) \equiv \alpha B(u) + \beta M(u) + \gamma D(u) + \delta X(u), \tag{15}$$

where $B(u)$ is the number of architectural boundaries touched, $M(u)$ is the number of modes or scenarios, $D(u)$ is the number of external dependencies, and $X(u)$ is the number of exceptions or special cases.

This quantity should be treated as a disciplined proxy rather than a precise estimate.

4.4. *Pre-Implementation Balance*

Replacing implementation with expected implementation potential, we obtain the pre-implementation analogue of SIB:

$$\text{SIB}^*(u) \equiv \frac{S(u)}{I^*(u) + \varepsilon}. \tag{16}$$

More importantly, we define an effective pre-implementation balance:

$$\text{SIB}^*_{\text{eff}}(u) \equiv \frac{S_{\text{eff}}(u)}{I^*(u) + \varepsilon}. \tag{17}$$

This metric characterizes how much *verifiable* intent is present relative to the expected cost of realizing it.

4.5. *Premise Balance*

Since specification may rest on assumptions that are themselves weakly grounded, we define a premise-oriented balance:

$$\text{PB}(u) \equiv \frac{P_{\text{eff}}(u)}{S(u) + \varepsilon}. \tag{18}$$

PB describes how much of the local intent is supported by verified or explicitly examined premises.

4.6. *Differential Pre-Implementation Balance*

As specification evolves, both explicit intent and expected realization cost may change. We therefore define a differential pre-implementation balance:

$$\text{dSIB}^*(u, t) \equiv \frac{\Delta S(u, t)}{\Delta I^*(u, t) + \varepsilon}. \tag{19}$$

This captures whether growing implementation expectations are matched by corresponding clarification of intent.

5. PROCESS AND STRUCTURAL OBSERVABILITY

Once specification evolves over time, and especially once code begins to emerge, balance alone is not sufficient. A second diagnostic layer is needed to characterize how the process behaves: whether intent is verifiable, whether changes create excessive review burden, how broadly impact propagates, and how stably the system returns to a satisfactory state.

5.1. *Review Pressure*

At the code-aware stage, let $\Delta S_{\text{impl}}(u,t)$ denote local implementation change volume, and let $\Delta N_{\text{spec}}(u,t)$ denote the change in intent-bearing specification units. We define Review Pressure as

$$\text{RVP}(u,t) \equiv \frac{\Delta S_{\text{impl}}(u,t)}{\Delta N_{\text{spec}}(u,t) + \varepsilon}. \tag{20}$$

This measures how much implementation change is being introduced per unit of clarified intent.

For the pre-implementation phase, the analogous quantity is

$$\text{SRP}(u,t) \equiv \frac{\Delta I^*(u,t)}{\Delta S(u,t) + \varepsilon}, \tag{21}$$

which measures how quickly expected realization cost is rising relative to the growth of explicit specification.

5.2. *Premise Load and Premise Density*

The absolute load of assumptions attached to a node is given by

$$\text{PL}(u) \equiv P(u). \tag{22}$$

A normalized view is Premise Density:

$$\text{PD}(u) \equiv \frac{P(u)}{S(u) + \varepsilon}. \tag{23}$$

These metrics help expose when local intent is supported by a large and potentially fragile network of assumptions.

5.3. *Probabilistic Friction*

Let $N_{\text{calls\_stable}}(u)$ denote the number of attempts, runs, or inference iterations needed until node u reaches a stable state. We define the Probabilistic Friction Index as

$$\text{PFI}(u) \equiv \frac{N_{\text{calls\_stable}}(u)}{|\mathcal{A}(u)| + \varepsilon}. \tag{24}$$

A cost-weighted variant is

$$\text{PFI}_C(u) \equiv \frac{C_{\text{inf\_stable}}(u)}{|\mathcal{A}(u)| + \varepsilon}. \tag{25}$$

PFI characterizes how many iterations are required to stabilize a unit of declared intent.

5.4. *Drift Velocity*

For any metric m, we define its drift velocity as

$$\text{DV}_m(u,t) \equiv \frac{d\, m(u,t)}{dt}. \tag{26}$$

In discrete form,

$$\text{DV}_m(u,t_k) \approx \frac{m(u,t_k) - m(u,t_{k-1})}{t_k - t_{k-1}}. \tag{27}$$

This allows longitudinal reasoning over balance, verifiability, friction, stability, and defect-related observables.

5.5. *Cognitive Load Proxy*

Local structural complexity may increase the practical cost of turning intent into stable implementation. We therefore define a cognitive load proxy:

$$\text{CLP}(u) \equiv \alpha\, \deg(u) + \beta\, B(u) + \gamma\, M(u), \tag{28}$$

where $\deg(u)$ is the local graph degree, $B(u)$ is the number of boundaries touched, and $M(u)$ is the number of modes or variants.

CLP is diagnostic rather than psychological: it approximates the structural density that a human or machine must navigate.

5.6. *Cognitive Load Gradient*

If nodes are embedded on a 2D map $x(u) \in \mathbb{R}^2$, we define a local gradient of cognitive load as

$$\text{CLG}(u) \equiv \sum_{v \in \mathcal{N}(u)} \frac{\text{CLP}(v) - \text{CLP}(u)}{\|x(v) - x(u)\| + \varepsilon}. \tag{29}$$

This quantity reflects how abruptly structural complexity changes around a node.

5.7. *Change Impact Radius*

Let $\text{dist}(u,v)$ denote graph distance in a chosen projection. We define the unweighted change radius as

$$\text{CR}_k(u) \equiv \left|\{v \in V : \text{dist}(u,v) \le k\}\right|. \tag{30}$$

A weighted version is

$$\text{WCR}_k(u) \equiv \sum_{\text{dist}(u,v) \le k} w(v), \tag{31}$$

where $w(v)$ may be chosen as $\text{CLP}(v)$, expected cost, local implementation mass, or another diagnostic proxy.

Together, these quantities characterize how widely a change or assumption can propagate.

5.8. *Checkpoint Stability*

Let, on a time window t, $N_{\text{regen}}(u,t)$ be the number of regenerations or restarts, $N_{\text{rollback}}(u,t)$ the number of rollbacks, $N_{\text{fail}}(u,t)$ the number of failed verification runs, and $N_{\text{steps}}(u,t)$ the total number of process steps. We define the Checkpoint Stability Index as

$$\text{CSI}(u,t) \equiv 1 - \frac{\alpha N_{\text{regen}}(u,t) + \beta N_{\text{rollback}}(u,t) + \gamma N_{\text{fail}}(u,t)}{N_{\text{steps}}(u,t) + \varepsilon}. \tag{32}$$

The weights α, β, γ allow a project to emphasise regenerations, rollbacks, or failures differently depending on their perceived severity.

We further define Time-to-Stable:

$$\text{TTS}(u) \equiv t^\star - t_0, \tag{33}$$

where $t^\star$ is the first time at which a chosen stability predicate holds.

5.9. *Specification Stability*

Before executable implementation exists, an analogous diagnostic is required. Let $N_{\text{reframe}}(u,t)$ be the number of specification reframings, $N_{\text{premise\_change}}(u,t)$ the number of changes to premises, $N_{\text{rejection}}(u,t)$ the number of product or stakeholder rejections, and $N_{\text{steps}}(u,t)$ the number of discussion or refinement steps. We define the Specification Stability Index as

$$\text{SSI}(u,t) \equiv 1 - \frac{\alpha N_{\text{reframe}}(u,t) + \beta N_{\text{premise\_change}}(u,t) + \gamma N_{\text{rejection}}(u,t)}{N_{\text{steps}}(u,t) + \varepsilon} \tag{34}$$

Again, the weights α, β, γ reflect the relative impact of reframings, premise changes, or explicit rejections.

Its temporal companion is Time-to-Stable-Spec:

$$\text{TTSS}(u) \equiv t^\star - t_0, \tag{35}$$

where $t^\star$ is the first time the specification is deemed sufficiently stable.

6. RETROSPECTIVE DEFECT METRICS

The previous sections describe a forward-facing observability stack. A third layer becomes necessary once a defect is discovered late in time: we may wish to formally identify how far back the defect originated, how much subsequent work was built upon it, and what balance conditions existed near its root.

This layer is retrospective by design. It does not measure current process state, but reconstructs latent cost and causal structure after a defect becomes visible.

6.1. *Defect Classes*

We distinguish two broad classes.

Implementation Defects are defects whose witness is executable: a test failure, violated invariant, broken behavior, failing integration, or other code-aware signal.

Value or Specification Defects are defects whose witness is semantic: a requirement reversal, stakeholder rejection, customer non-acceptance, or discovery that a key premise or concept was unsound, even if the implementation was locally correct.

6.2. *Retrospective Structure*

Consider a linearized trajectory $v_1 \to v_2 \to \ldots \to v_n$. Suppose a defect is detected at v_n, but its root is determined to be v_j for some $j < n$. Then the interval $[v_j, v_n]$ defines the defective segment.

6.3. *Defect Depth*

For both implementation and specification defects we define the defect depth as

$$\text{DD} \equiv n - j. \tag{36}$$

The value of DD counts how many steps or nodes separate the earliest point at which the defect could have been detected from the point at which it was actually detected. In implementation defects this root often corresponds to the first committed change that broke an invariant; in value defects it may be the earliest introduction of a contested premise.

6.4. *False Progress Mass*

Let $I(v_k)$ denote the implementation mass attached to node v_k. We define False Progress Mass as

$$\text{FPM} \equiv \sum_{k=j+1}^{n} I(v_k). \tag{37}$$

FPM represents the amount of implementation work performed after the root defect had already entered the trajectory.

6.5. *Rework Effort (Rework Cost)*

Let $\rho \in [0,1]$ denote the salvage ratio: the fraction of post-root work that survives after correction. We define Rework Effort (equivalently, rework cost) as

$$\text{RE} \equiv \text{RWC} \equiv (1-\rho) \cdot \text{FPM}. \tag{38}$$

6.6. *Retro Verification Overhead*

Let the fix window following defect discovery incur additional stabilization attempts, failures, or rollbacks. We define

$$\text{RVO} \equiv \Delta N_{\text{calls\_stable}} + \Delta N_{\text{fail}} + \Delta N_{\text{rollback}}. \tag{39}$$

A normalized form is

$$\text{RVO}_N \equiv \frac{\text{RVO}}{\text{FPM} + \varepsilon}. \tag{40}$$

6.7. *Defect Cost*

A compact retrospective cost observable is

$$\mathrm{DC} \equiv \mathrm{RWC} + \lambda \cdot \mathrm{RVO}, \tag{41}$$

where λ maps additional verification effort into a common cost scale.

6.8. *Defect Balance at Root*

To connect retrospective defects back to the forward observability stack, we define the Defect Balance at Root as

$$\mathrm{DBR} \equiv \mathrm{SIB}_{\mathrm{eff}}(v_j), \tag{42}$$

or, in settings where the implementation-aware local balance is not yet available,

$$\mathrm{DBR}^* \equiv \mathrm{SIB}^*_{\mathrm{eff}}(v_j). \tag{43}$$

These quantities make it possible to ask whether expensive future defects tend to emerge from distinctive balance regimes.

6.9. *Defect Density Relative to Specification*

At project or subsystem level, we may define

$$\mathrm{DDS}(u,t) \equiv \frac{N_{\mathrm{defects}}(u,t)}{S(u,t)+\varepsilon}, \tag{44}$$

or alternatively

$$\mathrm{DDS}_{\mathrm{eff}}(u,t) \equiv \frac{N_{\mathrm{defects}}(u,t)}{S_{\mathrm{eff}}(u,t)+\varepsilon}. \tag{45}$$

This reflects how often defects are observed relative to the local mass of explicit intent.

6.10. *Value and Premise Defects*

Not all late defects correspond to broken implementation. A feature can be implemented correctly yet still be conceptually rejected. In such cases, the defect root should be traced through assumptions and specification premises rather than through executable failure.

Let $\mathcal{P}_\times$ denote the set of contested premises identified after a value defect is discovered at v_n. If $\Delta\mathcal{P}(v_k)$ denotes newly introduced premises at step k, the earliest premise root is

$$j = \min\{k \le n : \Delta\mathcal{P}(v_k) \cap \mathcal{P}_\times \neq \varnothing\}. \tag{46}$$

For such defects we also consider the following quantities.

Defect Specification Cost—Let $S(v_k)$ denote the local specification signal. We define the defect specification cost as

$$\mathrm{DSC} \equiv \sum_{k=j}^{n} S(v_k), \tag{47}$$

representing the mass of specification built on a defective premise or concept.

Defect Premise Cost—Similarly, the mass of premises implicated in the defective path is

$$\mathrm{DPC}_P \equiv \sum_{k=j}^{n} P(v_k). \tag{48}$$

Defect Potential Cost—Even if code has not yet been written, a late-discovered value defect may already have accumulated substantial expected realization cost. We therefore define

$$\mathrm{DPC} \equiv \sum_{k=j}^{n} I^*(v_k), \tag{49}$$

the Defect Potential Cost.

Defect Premise Balance at Root—To connect value defects back to the balance stack, we define

$$\mathrm{DPBR} \equiv \mathrm{PB}(v_j), \tag{50}$$

which characterizes how well-supported the root premises were at the origin of the defective trajectory.

Value and Premise Defect Densities—We define value defect density as

$$\mathrm{VDD}(u,t) \equiv \frac{N_{\mathrm{value\_defects}}(u,t)}{S(u,t)+\varepsilon}, \tag{51}$$

or, using effective specification,

$$\mathrm{VDD}_{\mathrm{eff}}(u,t) \equiv \frac{N_{\mathrm{value\_defects}}(u,t)}{S_{\mathrm{eff}}(u,t)+\varepsilon}. \tag{52}$$

Premise defect density is

$$\mathrm{PDD}(u,t) \equiv \frac{N_{\mathrm{premise\_defects}}(u,t)}{P(u,t)+\varepsilon}. \tag{53}$$

7. INTERPRETIVE SUMMARY

The framework now consists of three logically connected layers.

First, the base SIB layer establishes where intent resides relative to implementation and what economic cost is incurred while converting intent into functionality.

Second, the pre-implementation and process layers extend this view to situations where code has not yet been written, and to dynamic questions of verification pressure, friction, cognitive load, stability, and propagation.

Third, the retrospective defect layer closes the loop by asking: when a future defect becomes visible, what earlier balance condition, premise structure, or specification trajectory allowed it to emerge?

The framework is therefore longitudinal. It begins before code, continues through intent-to-implementation conversion, and remains meaningful after failures are discovered.

8. DISCUSSION

These metrics are explicitly non-normative. They are not intended for optimization, ranking, or performance evaluation. Instead, they provide a structured lens for analyzing semantic allocation, realization cost, process stability, and retrospective defect structure in software systems.

An important consequence is that not all defects must be treated as implementation failures. Some are failures of assumptions, specification framing, or product-value alignment. By admitting these classes explicitly, the framework can reason about late reversals even when executable code is locally correct.

Over time, retrospective defect analysis may also support empirical discovery. If expensive defects cluster around certain balance regimes or weak premise support conditions, then balance metrics cease to be merely descriptive and become a basis for anticipatory diagnostics. Such use, however, remains observational rather than normative: the framework can identify recurrent patterns without prescribing a universal target value.

9. CONCLUSION

We argue that AI-assisted software development requires a shift from code-centric metrics toward intent-centric observability. Specification–Implementation Balance offers a compact formalism for reasoning about this transition.

Its extensions proposed here show that the same logic can be carried into three adjacent domains: pre-implementation analysis before code exists, process and structural diagnostics while intent is being realized, and retrospective defect analysis after failures or conceptual reversals are discovered.

Taken together, these layers form a coherent observability framework for AI-mediated software construction.

10. RELATION TO EXISTING METRICS

The metrics introduced in this work are not intended to replace existing software engineering metrics. Instead, they operate at a different abstraction level, focusing on semantic and economic mediation between specification, assumptions, implementation, and observable system behavior.

10.1. *Function Points*

Function Points (FP) measure externally observable functionality from the perspective of a system user, independent of implementation details. They are commonly used for cost estimation and productivity analysis.

In our framework, Functional Units (N_{func}) play a role analogous to Function Points, but with two important differences. First, Functional Units are treated as a *proxy* for observable behavior rather than as a standardized accounting unit. Second, we explicitly avoid interpreting N_{func} as a productivity measure.

The Functional Cost metric,

$$\text{FC} \equiv \frac{C_{\text{inf}}}{N_{\text{func}}}, \tag{54}$$

can be viewed as an AI-era analogue of FP-based cost models. However, unlike classical FP analysis, FC is not intended to support estimation or optimization, but rather to expose the economic footprint of AI-mediated realization.

10.2. *Requirements Traceability*

Requirements traceability aims to establish explicit links between requirements, design artifacts, code, and tests. Traceability matrices and coverage ratios are commonly used to ensure completeness and compliance, particularly in safety-critical systems.

Our framework does not attempt to enforce or reconstruct explicit trace links. Instead, it introduces aggregate and local observables that reflect the *distribution of intent* and the stability of its realization. The Specification–Implementation Balance,

$$\text{SIB} \equiv \frac{N_{\text{spec}}}{S_{\text{impl}}}, \tag{55}$$

can be interpreted as a coarse-grained signal of traceability drift: significant changes in SIB over time may indicate that intent is migrating toward implicit representation in code or, conversely, being externalized into specifications.

Similarly, the retrospective quantities DBR and DBR* make it possible to ask not merely whether a requirement was implemented, but under what earlier balance conditions future defects emerged.

Thus, while traceability answers the question *"Is this requirement implemented?"*, SIB addresses the orthogonal question *"Where does the system's meaning primarily reside?"*, and defect-root metrics extend this to *"Under what structural conditions do future failures emerge?"*

10.3. *Test and Coverage Metrics*

Code coverage metrics (e.g., line, branch, or path coverage) quantify the extent to which implementation is exercised by tests. In behavior-driven or test-driven development, tests themselves often act as executable specifications.

The Intent Yield metric,

$$\text{IY} \equiv \frac{N_{\text{func}}}{N_{\text{spec}}}, \tag{56}$$

is complementary to coverage metrics. Coverage measures how much of the implementation is validated, whereas Intent Yield measures how effectively declared intent materializes as observable behavior.

The Spec Verifiability Coverage metric,

$$\mathrm{SVC}(u) \equiv \frac{N_{\mathrm{ver}}(u)}{|\mathcal{A}(u)|}, \tag{57}$$

extends this logic to the specification side. It characterizes how much of explicit intent is actually bound to some operational check or validation channel.

Importantly, a system may exhibit high code coverage while still showing low effective balance or weak premise support, indicating that executable artifacts are being validated more thoroughly than the underlying intent or assumptions that motivated them.

10.4. *Architectural and Change Metrics*

Architectural metrics often focus on coupling, dependency count, modularity, or change propagation. Our structural observables such as CLP, CLG, CR_k, and WCR_k are closest in spirit to this family.

The difference is that these quantities are explicitly embedded within an intent-centric diagnostic stack. They are not used as isolated structure scores, but as supporting observables that help explain how balance, verification, and defect emergence interact with system structure.

10.5. *Summary of Conceptual Differences*

Existing metrics predominantly operate within a single artifact domain (code, tests, architecture, or requirements). The framework proposed here is explicitly cross-artifact, cross-phase, and non-normative.

Rather than optimizing for higher or lower values, the proposed metrics are intended to support longitudinal observation and qualitative reasoning about system evolution in AI-assisted and specification-driven development contexts.

11. THREATS TO VALIDITY

As this work proposes a conceptual and diagnostic framework rather than an empirical evaluation, several limitations and threats to validity must be explicitly acknowledged.

11.1. *Construct Validity*

The proposed metrics rely on abstract quantities such as Intent Atoms (N_{spec}, $\mathcal{A}(u)$), Premise Atoms ($\mathcal{P}(u)$), Functional Units (N_{func}), and Expected Implementation Potential ($I^*(u)$), all of which are proxy representations rather than direct semantic truth.

Different teams or projects may adopt divergent decompositions of specifications into stories, criteria, assumptions, or decisions, leading to variability in absolute values.

To mitigate this threat, the framework explicitly avoids normative interpretation of metric values and focuses on relative changes and longitudinal analysis within a single project.

11.2. *Measurement Subjectivity*

The identification and counting of Functional Units may be influenced by architectural style, domain granularity, or interface conventions. Likewise, the decomposition of specification into Intent Atoms and Premise Atoms may differ across teams.

This framework therefore treats such counts as qualitative observables rather than standardized accounting units.

11.3. *Pre-Implementation Estimation Risk*

The pre-implementation extension depends on $I^*(u)$, an expected implementation potential. This quantity is inherently estimated rather than observed, and may be wrong in absolute terms.

Its usefulness therefore depends on consistent estimation discipline within a project rather than on global accuracy.

11.4. *Dependence on AI Tooling*

Inference-related metrics such as Intent Conversion Cost, Functional Cost, Probabilistic Friction Cost, and related stabilization quantities depend on external AI model providers, pricing schemes, tool invocation styles, and context management strategies.

Changes in model efficiency, pricing, or orchestration may alter absolute values without reflecting any intrinsic change in system structure.

11.5. *Retrospective Root Ambiguity*

Defect-root metrics assume that a meaningful root v_j can be identified. In practice, causal structure may be distributed rather than singular, and the true root may be ambiguous.

This is especially relevant for value defects, where conceptual rejection may emerge gradually rather than at a single point. For this reason, retrospective root observables should be interpreted as approximations to causal localization rather than exact forensic truth.

11.6. *External Validity*

The framework is primarily motivated by AI-assisted and specification-driven development workflows. Its ap-

plicability to traditional, fully human-authored development processes may be limited, particularly in contexts where specifications are informal or absent.

However, the framework does not assume a specific programming language, model architecture, or development methodology, suggesting that its core concepts may generalize across a wide range of software systems.

11.7. *Risk of Metric Misuse*

As with any quantitative metric, there exists a risk of misuse if the proposed quantities are reinterpreted as performance indicators or optimization objectives. Such use would directly contradict the diagnostic intent of the framework and may lead to behavior consistent with Goodhart's law.

We therefore emphasize that the proposed metrics are intended solely for observability and interpretive analysis, not for ranking, benchmarking, or incentive alignment.

12. QUESTIONS AND ANSWERS

In response to community feedback, this section collects several frequently asked questions and the corresponding clarifications. These questions highlight potential weaknesses of the proposed framework, and the answers explain how the current formulation attempts to address or mitigate them.

12.1. *Subjectivity of basic units*

Question. How can Intent Atoms, Premise Atoms, and Functional Units be treated as objective measurements when their identification is inherently subjective?

Answer. The framework deliberately treats these units as proxy observables rather than absolute truths. Different projects may decompose requirements differently; however, by working with relative ratios and time-series rather than absolute values, the framework focuses on internal consistency rather than cross-project comparison. Moreover, developing a reference vocabulary and taxonomy of intent and premise atoms is identified as future work to improve comparability.

12.2. *Arbitrary weighting coefficients*

Question. Many metrics rely on coefficients $(\alpha, \beta, \gamma, \delta, \lambda)$ that appear arbitrary. How should these be chosen, and does this undermine the metrics?

Answer. The coefficients are intentionally not fixed across all contexts. They serve to modulate the relative importance of different factors, such as regenerations versus rollbacks, or structural boundaries versus modes. The framework acknowledges the need for empirical calibration: as more data accumulates, projects can derive *reference coefficients* suitable for common patterns. Until then, the coefficients should be treated as tunable parameters rather than hidden constants.

12.3. *Contamination by AI economics*

Question. Metrics such as Intent Conversion Cost or Functional Cost depend on external factors like language-model pricing, risking misleading signals. How is this handled?

Answer. Economic observables must be versioned or indexed by the underlying inference pricing and architecture. Comparing values across projects or over time requires normalizing to a common pricing baseline or reporting cost bands along with the raw figures. The framework stresses that economic metrics are for diagnostic awareness, not for cross-project benchmarking, precisely because external market changes can dominate absolute values.

12.4. *Linear defect tracebacks*

Question. Defects in complex systems often arise from distributed causes rather than a single "root" node on a linear path. Is the linear defect model too simplistic?

Answer. The linearization used in retrospective analyses serves as a pragmatic projection: it collapses a potentially multi-branched history into a sequence so that a "first moment" of invalidity can be identified. This does not deny that causality may be distributed. Rather, it isolates the earliest point at which an assumption or change allowed the later defect to emerge. Future work could explore fully graph-based tracebacks, but the current linear model already improves over informal root-cause narratives by making assumptions explicit.

12.5. *Non-normative metrics and Goodhart's law*

Question. If these metrics are exposed to management, won't teams start optimising for them, undermining their diagnostic value?

Answer. The framework is designed to measure latent properties—such as balance or premise support—rather than productivity. While any metric can be gamed if tied to incentives, the intended use is analytic rather than evaluative. Teams can collect these signals without attaching targets, using them to understand and predict structural risks rather than to rank individuals. This diagnostic orientation is emphasised repeatedly in the text.

12.6. *Estimating expected implementation potential I^**

Question. The pre-implementation balance replaces real code mass with an estimated potential (I^*), which

may be unreliable. Does this undermine the pre-implementation metrics?

Answer. Estimation is indeed uncertain, and the framework does not deny this. However, using structured proxies such as counts of boundaries, modes, dependencies, or exception cases provides a more disciplined approximation than unstructured guesswork. As with weighting coefficients, the estimation of I^* should be empirically refined. The pre-implementation metrics are most valuable for capturing relative change (growth of expected complexity against growth of intent), rather than for predicting absolute effort.

13. THREATS TO VALIDITY (ADDITIONAL COMMENTS)

The critique presented earlier raises several methodological concerns about subjectivity, weight calibration, economic contamination, linear causal models, metric misuse, and estimation error. Many of these issues are acknowledged in the original threats-to-validity section. Rather than treating them as fatal flaws, we view them as points of caution and directions for future refinement. The framework explicitly avoids normative interpretation of metric values and treats all quantities as diagnostic signals rather than targets. It is the accumulation of data and the development of shared vocabularies and calibration procedures that will determine the practical utility of these observables.